Google Summer of Code 2026

Implement Model Context Protocol (MCP) Support for MetaCall Deploy & FaaS

Swastik Prakash

B.Tech Computer Science Engineering

Jaypee Institute of Information Technology, Noida

swastikprakashofficial@gmail.com

github.com/Swastik-Prakash1 | linkedin.com/in/swastik-prakash

+91 8081605787 · Uttar Pradesh, India

Organization:	MetaCall	**Project:**	#8 – MCP for Deploy & FaaS
Size:	Medium (175 hours)	**Difficulty:**	Medium
Mentors:	Mostafa Wael Kamal, Jose Antonio Dominguez		

Contents

1 Synopsis **3**

2 About Me **4**
- 2.1 Personal Information . . . 4
- 2.2 Education . . . 4
- 2.3 Competitive Programming and DSA . . . 4
- 2.4 Hackathon Achievement . . . 5
- 2.5 Technical Skills . . . 5
- 2.6 Certifications . . . 5
- 2.7 Selected Projects . . . 5
  - 2.7.1 MediMind – AI Clinical Triage (1st Place, MLH Hackathon) . . . 5
  - 2.7.2 Should I? – AI Trust Analysis System . . . 6
  - 2.7.3 AI Interview Assistant . . . 6
  - 2.7.4 Streaming-ML – Algorithm Comparison for LLMs . . . 6

3 Pre-GSoC Contributions to MetaCall **7**
- 3.1 PR #200 – metacall/deploy: Interactive Screen UX Fix . . . 7
- 3.2 MCP Server Proof-of-Concept . . . 7
- 3.3 MCP Server Proof-of-Concept . . . 9
  - 3.3.1 Architecture . . . 9
  - 3.3.2 Three Implemented MCP Tools . . . 9
  - 3.3.3 Live Demo Transcript . . . 10
- 3.4 Bug Investigation in metacall/faas . . . 10

4 Project Understanding **12**
- 4.1 What Is MetaCall? . . . 12
- 4.2 MetaCall Ecosystem . . . 12
- 4.3 What Is MCP? . . . 12
- 4.4 Why This Combination Matters . . . 12
- 4.5 FaaS API Endpoints . . . 13

5 Project Proposal **14**
- 5.1 Problem Statement . . . 14
- 5.2 Proposed Solution . . . 14
- 5.3 Full Architecture . . . 14
- 5.4 Repository Structure . . . 15
- 5.5 MCP Tools: Complete Specification . . . 16
- 5.6 Key Implementation Details . . . 16
  - 5.6.1 Dual Transport Support . . . 16
  - 5.6.2 Repository Deployment Tool . . . 17
  - 5.6.3 MCP Resources . . . 17

5.6.4 End-to-End Request Flow 18

6 Detailed Timeline (175 Hours) 19

7 Benefits to the Community 21

7.1 For MetaCall 21

7.2 For the Open Source Community 21

7.3 For Developers 21

8 Related Work and Differentiation 21

9 Why I Am the Right Person for This 22

10 Availability and Commitments 22

11 AI Usage Disclosure 23

12 Conclusion 23

1 Synopsis

MetaCall is the only open-source polyglot runtime that lets developers call functions across Python, JavaScript, TypeScript, Go, Rust, and more, all seamlessly. This project makes MetaCall's FaaS platform **AI-native** by building a production-grade **Model Context Protocol (MCP) server**. Once complete, Claude, GitHub Copilot, and any MCP-compatible AI assistant will be able to deploy, list, inspect, and call serverless functions through plain English, right from VS Code or any AI interface.

This is not a theoretical proposal. I have **already built and demonstrated** a working proof-of-concept MCP server that connects VS Code GitHub Copilot to a local `metacall/faas` instance. I demoed it live to MetaCall founder **viferga**, who responded: *"awesome mate."* Mentor **Mostafa Wael Kamal** has also reviewed my work. This GSoC project takes that working foundation and turns it into a production-grade, tested, documented, and officially integrated tool for the entire MetaCall ecosystem.

MetaCall Core has 1,770+ GitHub stars, is written in C, supports 18 language ecosystems, and is licensed under Apache 2.0. MetaCall FaaS is the TypeScript reimplementation powering `dashboard.metacall.io` for local and cloud serverless deployments. MetaCall Protocol provides shared TypeScript types across the ecosystem. My MCP server sits on top of all three, giving AI assistants a standardized way to interact with this entire stack.

2 About Me

2.1 Personal Information

Full Name	Swastik Prakash
Email	swastikprakashofficial@gmail.com
GitHub	github.com/Swastik-Prakash1
LinkedIn	linkedin.com/in/swastik-prakash
Phone	+91 8081605787
Location	Uttar Pradesh, India
University	Jaypee Institute of Information Technology, Noida
Degree	B.Tech, Computer Science Engineering (Expected 2027)
Year	3rd year
Timezone	IST (UTC+5:30)

2.2 Education

I am a 3rd-year B.Tech CSE student at JIIT Noida. My coursework includes Machine Learning, Artificial Intelligence, Big Data Analytics, Data Structures, and Algorithms. I consistently apply classroom learning to real systems, whether it is building AI triage assistants, deploying NLP pipelines, or solving algorithmic challenges under competitive constraints.

2.3 Competitive Programming and DSA

> I have been deeply invested in Data Structures and Algorithms since my second semester of first year. It started as curiosity and quickly became the core of how I think about software. Today I am **ranked 3rd in my entire college in DSA** and hold the **2nd position in AI/ML** across all departments.

I maintain a dedicated GitHub repository where I archive well-thought-out competitive programming solutions covering topics from graph algorithms and dynamic programming to advanced data structures like segment trees, tries, and balanced BSTs. Each solution is annotated with the approach, complexity analysis, and edge cases I considered.

I have been practicing competitive programming consistently for over two years now, and I plan to keep going. I regularly solve problems on Codeforces, LeetCode, and CodeChef, and I actively prepare for technical interviews because I genuinely enjoy the process of breaking down hard problems into clean, efficient code. I have won multiple DSA competitions at the college level and participate actively in inter-college coding contests.

This background directly benefits my work on the MCP server. When I built the proof-of-concept, the zip-and-upload pipeline for `deploy_local` required careful stream handling,

buffering, and async coordination. My instinct to think about edge cases and resource management comes straight from years of competitive problem solving.

2.4 Hackathon Achievement

1st Place – Build with Gemini × MLH, Zetech HackDay (International, 2026). Won a 12-hour AI hackathon powered by Google Gemini and Major League Hacking with **MediMind**, my AI Clinical Triage Assistant. Built a full multimodal system with persistent session memory, vision model inference, context-aware multi-turn reasoning, and emergency SOS Guardian Mode. [Devpost]

2.5 Technical Skills

Domain	Skills and Tools
Languages	Python, TypeScript/JavaScript, C++
DSA / CP	Algorithm Design, Graph Theory, DP, Segment Trees, Tries, Competitive Programming
AI / ML	LLM Integration, RoBERTa, OpenAI API, Hugging Face, Transformers, Multimodal Pipelines
Backend	Flask, REST API design, Node.js v22, npm v10
DevOps	Docker, GitHub Copilot (agent mode), VS Code, Git, GitHub Actions
Cloud	Google Cloud Platform, Oracle Cloud Infrastructure
Protocols	MCP, HTTP, JSON, stdio/SSE transports

Table 1: Technical skills

2.6 Certifications

- Oracle Certified AI Foundations Associate (OCI)
- Google Cloud Study Jam, Google Developer Student Clubs
- CDAC Noida, Bootcamp on Cloud Computing (FutureSkills PRIME)

2.7 Selected Projects

2.7.1 MediMind – AI Clinical Triage (1st Place, MLH Hackathon)

Built a production-grade multimodal AI triage system with persistent session memory, vision model inference, and context-aware multi-turn clinical reasoning. Engineered a real-time SOS Guardian Mode with emergency override, and implemented clinical SOAP report generation for structured specialist handoff. *Stack: Python, Flask, Google Gemini, Multimodal Vision.*

2.7.2 Should I? – AI Trust Analysis System

Built a scalable fake review detection system using fine-tuned RoBERTa for real-time inference across 1,000+ reviews per product, with a deterministic trust-scoring engine that aggregates linguistic and behavioral anomalies. *Stack: Python, RoBERTa, NLP, Flask.*

2.7.3 AI Interview Assistant

Developed semantic similarity scoring using embeddings for rubric-aligned response evaluation. Integrated speech signal processing and visual features into a multimodal assessment framework. *Stack: Python, NLP, Speech Processing, ML.*

2.7.4 Streaming-ML – Algorithm Comparison for LLMs

A research-oriented repository comparing different streaming algorithms for large language models, combining competitive algorithm analysis with practical ML engineering. *Stack: Python, ML Algorithms, Benchmarking.*

3 Pre-GSoC Contributions to MetaCall

I did not wait for GSoC to start contributing. Before writing this proposal, I explored the codebase, submitted a pull request, found and documented real bugs, and built a working MCP proof-of-concept that earned praise from MetaCall's founder.

3.1 PR #200 – metacall/deploy: Interactive Screen UX Fix

Issue: #154, "feat: Any interactive screen should show: Ctrl+C for cancel"

Status: Open PR (draft), reviewed positively, awaiting merge.

Evidence Links – PR #200

- **PR conversation (Discord):** https://discord.com/channels/781987805974757426/1242205848453775412/1478742776773611712
- **PR result / maintainer feedback (Discord):** https://discord.com/channels/781987805974757426/1242205848453775412/1478746380418678867
- **Pull Request (GitHub):** https://github.com/metacall/deploy/pull/200

3.2 MCP Server Proof-of-Concept

Repository: https://github.com/Swastik-Prakash1/metacall-mcp

Evidence Links – MCP Proof-of-Concept

- **MCP working demo video (Discord):** https://discord.com/channels/781987805974757426/1242205848453775412/1479423737110724651
- **MCP PR / discussion post (Discord):** https://discord.com/channels/781987805974757426/1242205848453775412/1479329899180196011

Before submitting, I read the original closed PR #173 and studied maintainer **viferga's specific rejection feedback** line by line to make sure my implementation addressed every concern.

What I changed:

- **Persistent screens** (`inspect.ts`, `logs.ts`): Added a dimmed "Press Ctrl+C to cancel." hint inside the infinite/polling loops, exactly where viferga said it belongs.
- **List-type prompts** (`selection.ts`): Added an "Exit" option instead of Ctrl+C, following viferga's feedback that list prompts should offer a clean graceful exit.

The logic for both patterns as a flowchart:

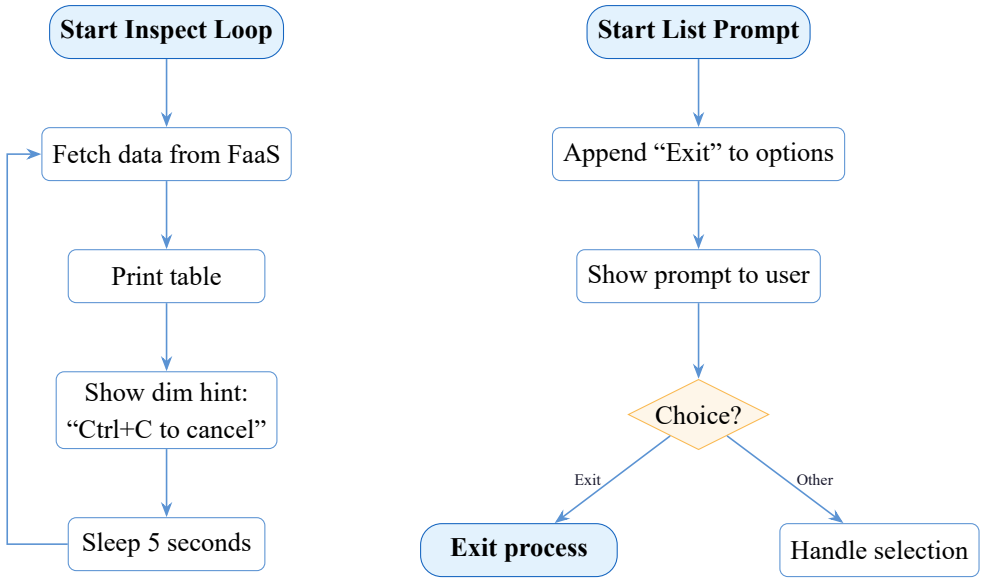

Figure 1: Two UX patterns: persistent screen loop (left) and list prompt with Exit option (right)

The TypeScript implementation:

```
p.printTable();
process.stdout.write(
  '\n\x1b[2m  Press Ctrl+C to cancel.\x1b[0m\n'
);
await sleep(5000);
```

Listing 1: inspect.ts – Ctrl+C hint

```
const EXIT_OPTION = 'Exit';
const handleExit = (value: string): void => {
  if (value === EXIT_OPTION) {
    process.stdout.write('\nExiting...\n');
    process.exit(0);
  }
};
```

Listing 2: selection.ts – Exit option

Testing: Ran local FaaS with `docker compose up`, then `node dist/index.js --dev --inspect`. Confirmed the dimmed hint appeared. Sent a screenshot to viferga on Discord.

3.3 MCP Server Proof-of-Concept

Repository: github.com/Swastik-Prakash1/metacall-mcp

Status: Fully working. Demoed via video to viferga, who said **"awesome mate."** Shared with mentor Mostafa Wael Kamal.

I built a TypeScript MCP server that connects VS Code GitHub Copilot to a local `metacall/faas` Docker instance. A developer types plain English in Copilot Chat and can deploy, list, or call polyglot serverless functions without touching a terminal.

3.3.1 Architecture

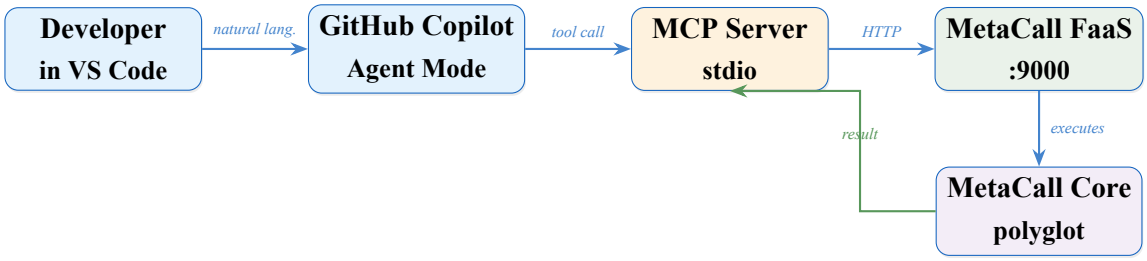

Figure 2: PoC architecture: Copilot to MetaCall FaaS via MCP

3.3.2 Three Implemented MCP Tools

Tool	What it does
`list_deployment`	Calls `GET /api/inspect`, returns all deployed functions with status, version, prefix
`deploy_local`	Zips directory, uploads via `POST /api/package/create`, deploys via `POST /api/deploy/create`
`call_function`	Calls the function endpoint, returns the result

Table 2: MCP tools in the proof-of-concept

The deploy workflow as a flowchart:

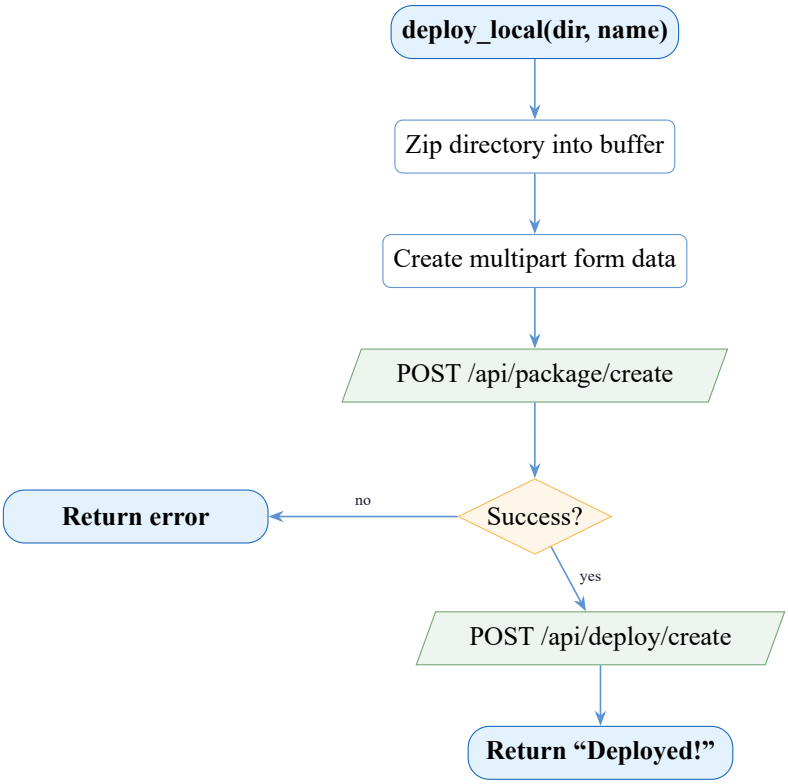

Figure 3: Flowchart: deploy_local tool workflow

3.3.3 Live Demo Transcript

Live Demo: Copilot Chat with metacall-mcp

`User:` list my metacall deployments
`Copilot:` No deployments found.

`User:` deploy the directory C:\D\Pro\mc\hello-test as hello-test
`Copilot:` Deployed successfully.

`User:` list my metacall deployments
`Copilot:` hello-test (ready), version: v1, prefix: f889f5dee176

`User:` call the hello function with name World
`Copilot:` "Hello World from MetaCall!"

```python
# main.py
def hello(name):
    return f'Hello {name} from MetaCall!'
```

Listing 3: hello-test Python function deployed via MCP

3.4 Bug Investigation in metacall/faas

While setting up the local FaaS for testing, I discovered and documented two bugs:

1. **CRLF line endings in `test/test.sh`** causing `exec: not found` errors inside Linux Docker containers. Fixed by converting to LF and adding `.gitattributes`: `*.sh text eol=lf`.

2. **Race condition in `src/controller/package.ts`**: busboy's `finish` event fires before the async `fs.promises.mkdtemp` completes, causing `resource.blob` to be `undefined`. This blocks all package deployment tests. I documented the root cause and reported it.

4 Project Understanding

4.1 What Is MetaCall?

MetaCall is an extensible, embeddable, cross-platform polyglot runtime. It lets you call functions written in one programming language from another: Python from JavaScript, Node.js from Python, Rust from TypeScript, all transparently.

The core runtime (`metacall/core`) is written in C, has 1,770+ GitHub stars, 220 forks, and supports 18 language ecosystems. It is licensed under Apache 2.0.

4.2 MetaCall Ecosystem

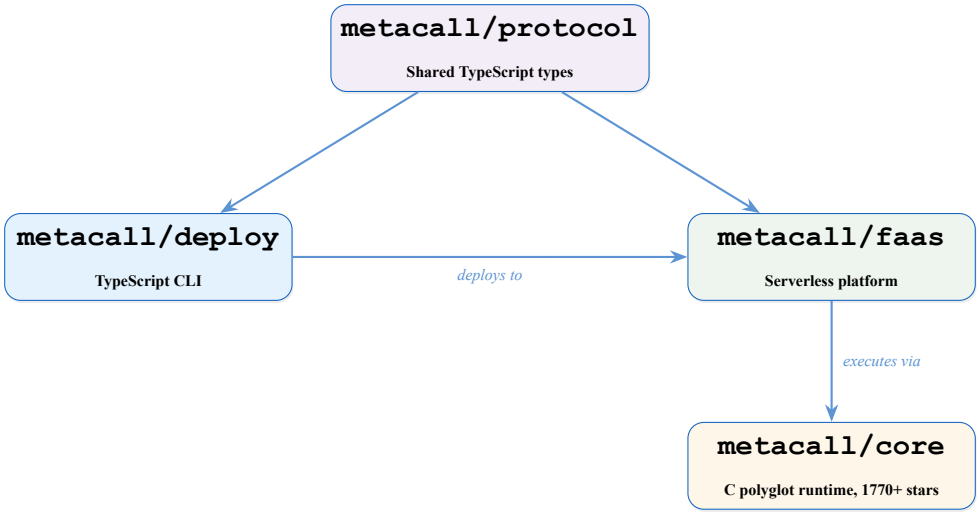

Figure 4: MetaCall ecosystem: protocol, deploy, faas, and core

4.3 What Is MCP?

The Model Context Protocol is an open standard created by Anthropic that lets AI assistants interact with external tools through a standardized interface. Think of it as a USB port for AI tools. An MCP server exposes three primitives:

- **Tools** are actions the AI can invoke ("deploy this function")
- **Resources** are data the AI can read (deployment metadata)
- **Prompts** are reusable templates for common workflows

4.4 Why This Combination Matters

MetaCall's FaaS is powerful, but right now it requires CLI commands or direct HTTP calls. MCP bridges that gap: developers interact with polyglot serverless functions through natural

language in any AI tool. The time from idea to deployed function drops from minutes to seconds.

This is exactly what the GSoC project description asks for: *"Implement a minimal project that can be used for Claude or similar LLMs from Visual Studio Code or Antigravity that allows to deploy functions, list the existing ones, or run them locally in multiple languages."*

4.5 FaaS API Endpoints

Method	Endpoint	Purpose
GET	/api/readiness	Health check
GET	/api/inspect	List all deployments
POST	/api/package/create	Upload a zip package
POST	/api/repository/add	Clone a git repository
POST	/api/deploy/create	Create a deployment
POST	/api/deploy/delete	Delete a deployment
POST	/api/deploy/logs	Fetch deployment logs

Table 3: MetaCall FaaS API endpoints

5 Project Proposal

5.1 Problem Statement

MetaCall's polyglot FaaS platform currently requires developers to use a CLI or raw HTTP requests. In 2026, where AI coding assistants are integral to every developer's workflow, there is no standard way to interact with MetaCall through an AI interface. Project #8 fixes this.

5.2 Proposed Solution

Build a production-grade MCP server in TypeScript that exposes MetaCall Deploy and FaaS capabilities as MCP tools, resources, and prompts. This enables any MCP-compatible AI assistant (Copilot, Claude, Antigravity) to interact with MetaCall through natural language.

5.3 Full Architecture

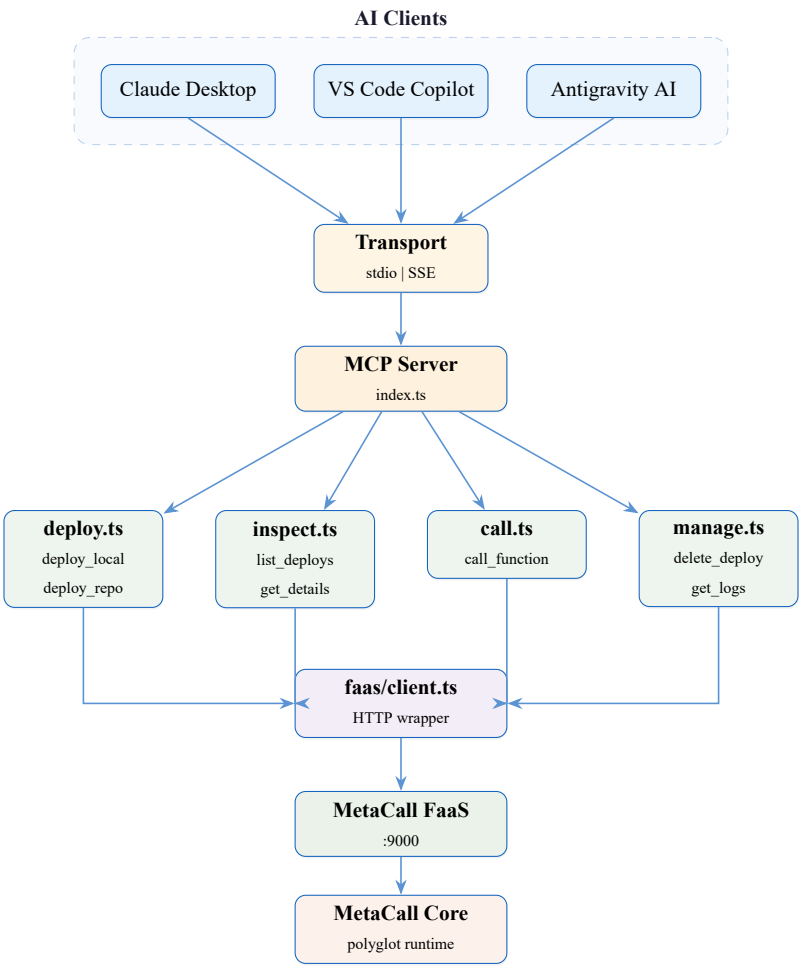

Figure 5: Full production architecture of the MCP server

5.4 Repository Structure

```
metacall-mcp/
  src/
    index.ts              # MCP server entry point
    tools/
      deploy.ts           # deploy_local, deploy_repository
      inspect.ts          # list_deployments, get_details
      call.ts             # call_function
      manage.ts           # delete_deployment, get_logs, env
    transport/
      stdio.ts            # VS Code, Claude Desktop
      sse.ts              # Antigravity, web-based tools
    faas/
      client.ts           # HTTP client for all FaaS APIs
      types.ts            # Types (uses metacall/protocol)
    resources/
      deployments.ts      # MCP Resources: deployment data
    prompts/
      templates.ts        # MCP Prompt templates
  tests/integration/
    deploy.test.ts        # Jest tests for deploy tools
    inspect.test.ts       # Jest tests for inspect tools
    call.test.ts          # Jest tests for call tools
  docs/
    README.md             # Setup, configuration, usage
    INTEGRATION.md        # AI client integration guide
  package.json
  tsconfig.json
```

Listing 4: Proposed final project structure

5.5 MCP Tools: Complete Specification

Tool	Description	Phase
`list_deployments`	List all deployments with status, version, prefix	1
`deploy_local`	Deploy from a local directory path	1
`call_function`	Call any deployed function with arguments	1
`deploy_repository`	Deploy from a GitHub repository URL	2
`get_function_deta`	Get function signature, parameters, types	2
`delete_deployment`	Delete a deployment by name	2
`get_logs`	Fetch logs for a specific deployment	2
`set_env_vars`	Set environment variables for a deployment	3

Table 4: All MCP tools across implementation phases

5.6 Key Implementation Details

5.6.1 Dual Transport Support

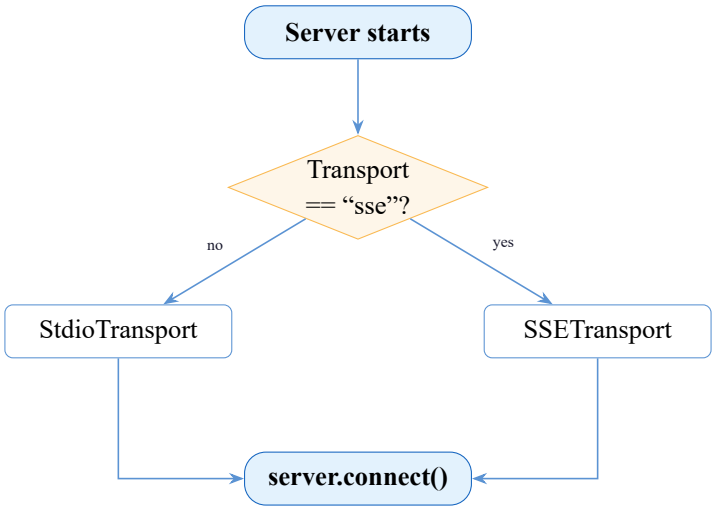

Figure 6: Flowchart: Transport selection logic

```
import { StdioServerTransport }
  from '@modelcontextprotocol/sdk/server/stdio.js';
import { SSEServerTransport }
  from '@modelcontextprotocol/sdk/server/sse.js';

const transport =
  process.env.MCP_TRANSPORT === 'sse'
    ? new SSEServerTransport('/sse', response)
    : new StdioServerTransport();

```

```
11 await server.connect(transport);
```

Listing 5: Transport selection in index.ts

5.6.2 Repository Deployment Tool

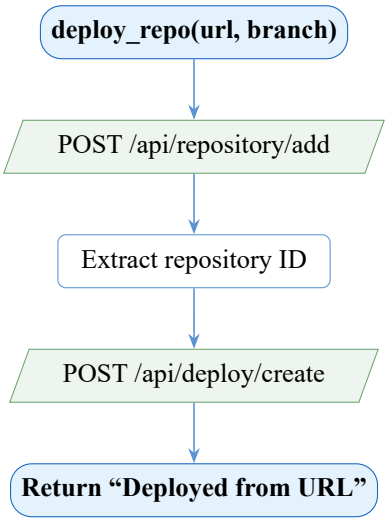

Figure 7: Flowchart: deploy_repository workflow

```
1  server.tool('deploy_repository', {
2    repoUrl: z.string()
3      .describe('GitHub repository URL'),
4    branch:  z.string().default('main'),
5    plan:    z.string().default('Essential')
6  }, async ({ repoUrl, branch, plan }) => {
7    const { id } = await faas.post(
8      '/api/repository/add',
9      { url: repoUrl, branch }
10   );
11   const result = await faas.post(
12     '/api/deploy/create',
13     { id, resourceType: 'repository', plan }
14   );
15   return {
16     content: [{
17       type: 'text',
18       text: `Deployed from ${repoUrl}`
19     }]
20   };
21 });
```

Listing 6: deploy_repository tool

5.6.3 MCP Resources

```
server.resource(
  'deployments',
  'metacall://deployments',
  async (uri) => {
    const data = await faas.get('/api/inspect');
    return {
      contents: [{
        uri: uri.href,
        mimeType: 'application/json',
        text: JSON.stringify(data, null, 2)
      }]
    };
  }
);
```

Listing 7: Deployment resource registration

5.6.4 End-to-End Request Flow

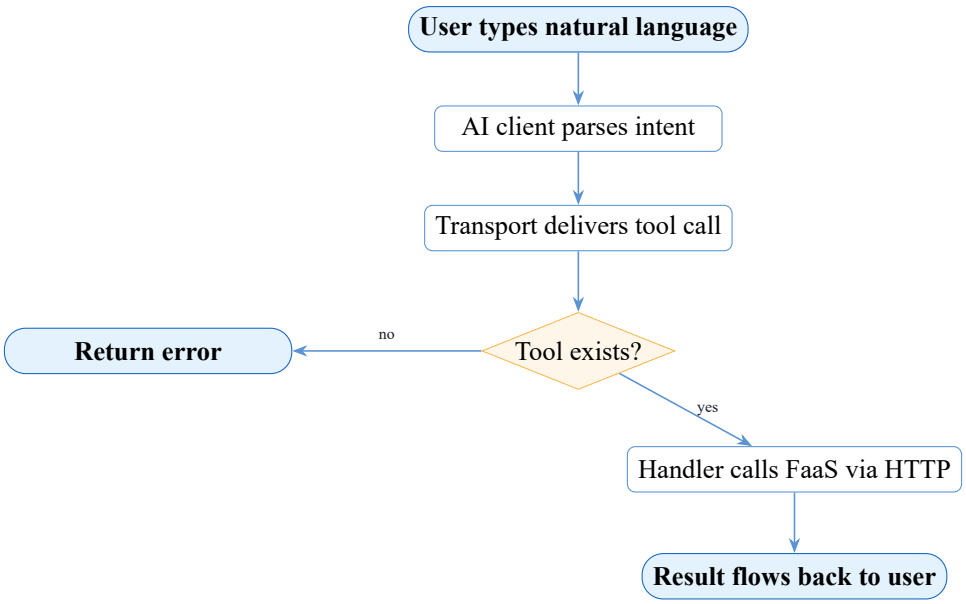

Figure 8: Flowchart: End-to-end MCP request lifecycle

6 Detailed Timeline (175 Hours)

Phase	Dates	Deliverables	Hrs
Bonding	May 8–26	Deep-dive into `metacall/protocol` types. Align architecture with mentors. Set up CI/CD. Write design doc.	20
Phase 1	May 27 – Jun 16	Core server scaffold. `list_deployments`, `deploy_local`, `call_function`. stdio transport. Basic README. Unit tests.	45
Phase 2	Jun 17 – Jul 14	`deploy_repository`, `get_function_details`, `delete_deployment`, `get_logs`. MCP Resources. SSE transport. Claude Desktop integration.	50
Midterm	Jul 14	Phase 1 and 2 working, tested, documented.	–
Phase 3	Jul 15 – Aug 4	Env variable support. MCP Prompt templates. Antigravity integration. npm publish setup. Integration tests with Jest.	40
Phase 4	Aug 5–25	Full docs (README, INTEGRATION guide). Official MetaCall docs integration. Demo video. Final PR review.	20
Total			**175**

Table 5: GSoC 2026 project timeline

Weekly rhythm: I plan to commit 25 to 30 hours per week. I will post weekly progress updates publicly, as agreed with MetaCall's mentorship process. No planned vacations during the program.

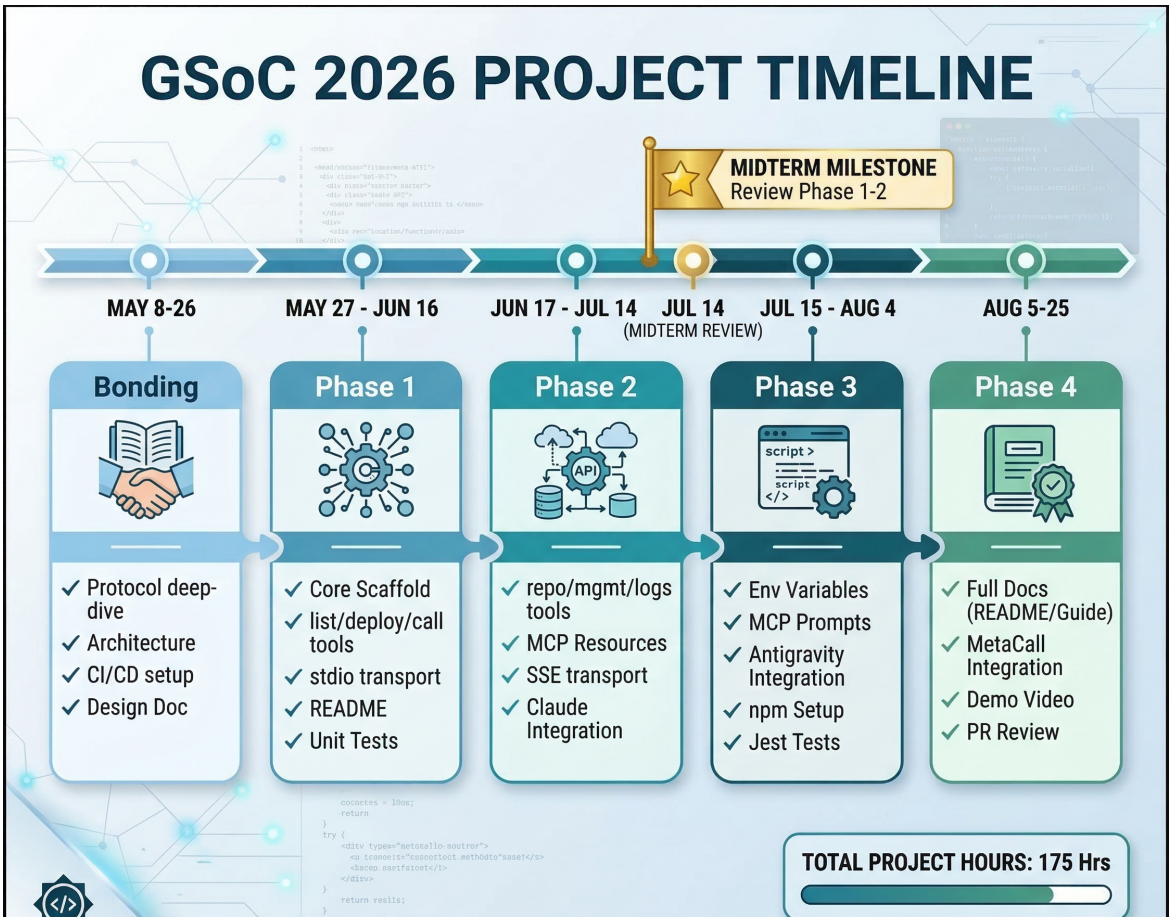

Figure 9: Project timeline snapshot

7 Benefits to the Community

7.1 For MetaCall

- Makes MetaCall **AI-native**: the first polyglot FaaS with full MCP support.
- Lowers the barrier to entry. Developers deploy polyglot functions without learning the CLI or API docs.
- Positions MetaCall as a leader at the intersection of AI tooling and polyglot computing.
- Directly fulfills every stated outcome of Project #8.

7.2 For the Open Source Community

- Published as an **npm package** for immediate use.
- Reference implementation of an MCP server wrapping a cloud API, useful for anyone learning MCP.
- Integrates with Antigravity, extending reach beyond VS Code.

7.3 For Developers

A developer will be able to type: *"Deploy my Python flask app from github.com/me/myapp and call the predict function with input 42"* and have it work. No CLI. No API docs. Just natural language.

8 Related Work and Differentiation

Project	What it does	How mine differs
AWS Lambda MCP	MCP for AWS Lambda	Not polyglot, not open source
Vercel AI SDK	AI tools for Vercel	No polyglot runtime support
My PoC	Working MCP for MetaCall	Only stdio, 3 tools, no tests
This project	**Full MCP server**	**8 tools, both transports, tested, docs, npm-published**

Table 6: Comparison with existing work

9 Why I Am the Right Person for This

> Most applicants write proposals. I wrote **code first**.

1. **Working proof-of-concept already built.** I have a functioning MCP server that delivers the core expected outcome of Project #8. Not theoretical.

2. **Mentor validation.** viferga (MetaCall founder) watched my demo and said "awesome mate." Mostafa Wael Kamal (Project #8 mentor) has reviewed my work.

3. **I know the codebase.** I read `metacall/deploy`, `metacall/faas`, and `metacall/protocol`. I found two undocumented bugs. I submitted a PR following maintainer feedback.

4. **Strong algorithmic foundation.** Ranked 3rd in DSA and 2nd in AI/ML in my college. Two years of competitive programming.

5. **Proven hackathon winner.** 1st Place at Build with Gemini × MLH international hackathon with MediMind.

6. **Domain expertise.** AI/ML background (LLM integration, OpenAI API, multimodal systems) directly relevant to MCP.

7. **Realistic proposal.** The 175-hour timeline is based on hours already logged building the PoC and exploring the codebase.

8. **Community engagement.** Joined MetaCall Discord, shared work proactively, engaged with mentors technically.

10 Availability and Commitments

- **Hours per week:** 25–30 hours, meeting the 175-hour target comfortably
- **Timezone:** IST (UTC+5:30), overlap with European and Asian mentors
- **Academic conflicts:** Semester exams in early June; will notify mentors two weeks ahead and compensate hours in adjacent weeks
- **Communication:** Available daily on Discord; email response within 24 hours
- **Other commitments:** No other GSoC applications. This is my only, focused proposal
- **Internet:** Stable broadband, no disruptions

11 AI Usage Disclosure

In accordance with MetaCall's AI Usage Policy:

AI Usage Disclosure

AI tools (GitHub Copilot in agent mode) were used as writing and structuring assistants during drafting of this proposal. All technical descriptions, code snippets, architectural decisions, and contributions described here reflect my own work, understanding, and implementations. AI was not used to generate any code submitted to MetaCall repositories. Every line of code in the PoC and PR #200 was written, understood, and tested by me personally.

12 Conclusion

MetaCall is solving a genuinely hard problem: making polyglot computing feel as natural as single-language development. MCP support takes this one step further, making polyglot serverless computing as natural as typing a sentence to your AI assistant.

I have not just read about this project. I have built it, tested it, shipped a PR, found bugs, and earned the praise of the project's founder. I am ready to take this from proof-of-concept to a production-grade, documented, and officially integrated tool that benefits the entire MetaCall community.

I look forward to working with the MetaCall team this summer.

Swastik Prakash
swastikprakashofficial@gmail.com
github.com/Swastik-Prakash1